



北京航空航天大学
BEIHANG UNIVERSITY



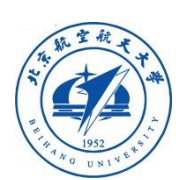
微机原理与接口技术

第八章

林 新

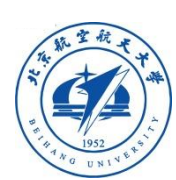
Lx@buaa.edu.cn

北京航空航天大学 自动化学院



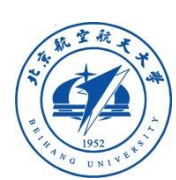
主要内容

- 中断的概念
- 中断的一般处理过程：中断请求、中断响应、中断处理、中断返回
- 中断源的识别
- 中断优先权的确定
- 8086系列微机的中断：分类、优先级、中断向量表、可屏蔽中断的响应过程
- 可编程中断控制器8259A简介。



基本要求

- 掌握中断的概念及中断处理过程
- 掌握8086系列微机的中断类型码、中断向量表与中断服务程序的入口地址三者的关系
- 了解中断服务程序的编写方法
- 了解可编程中断控制器8259A的结构



章节安排

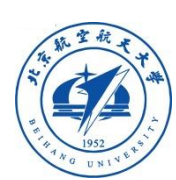
1. 中断概述
2. 8086中断系统
3. 中断服务程序设计
4. 8259A简介



8.1 中断概述

8.1.1 中断概念和分类

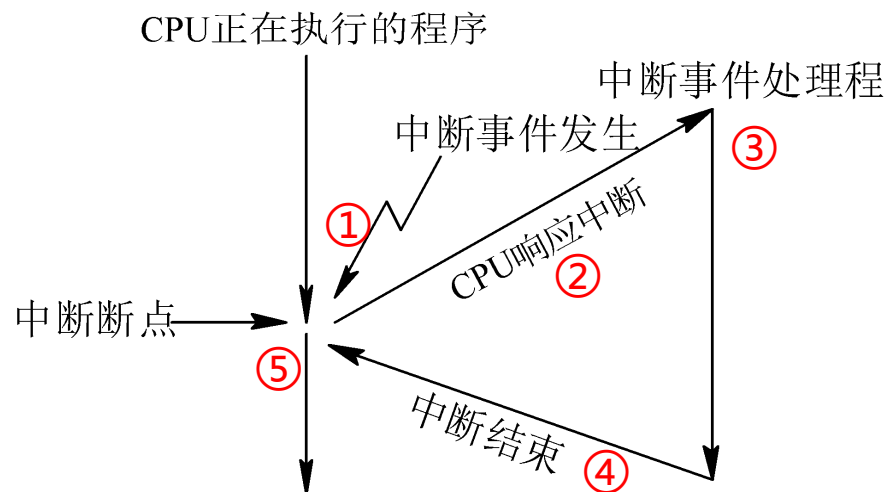
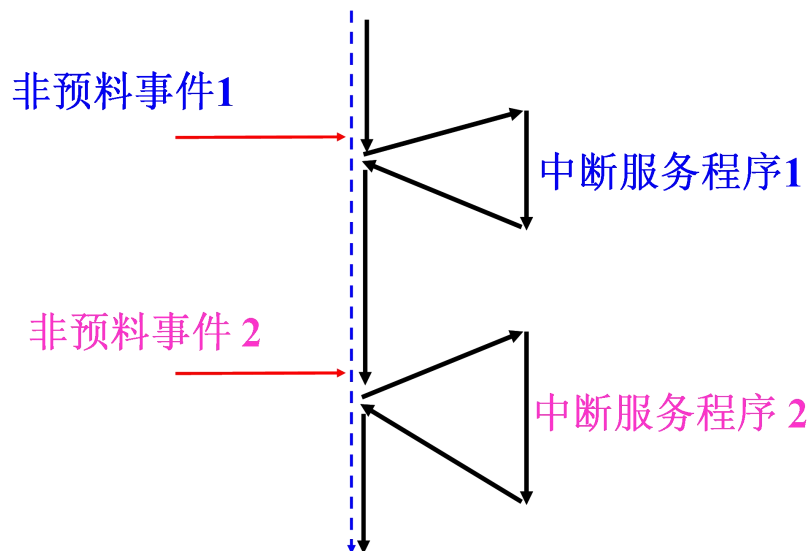
- 中断定义和功能
- 中断源和中断分类
- 中断向量表
- 中断优先级和中断嵌套



(1) 中断的定义和功能

- 在CPU正常运行程序时，由于内部或外部某个**非预料事件**的发生，使CPU暂停正在运行的程序，而转去执行处理引起中断事件的程序，然后再返回被中断了的程序继续执行。这个过程就是**中断**

CPU执行流程



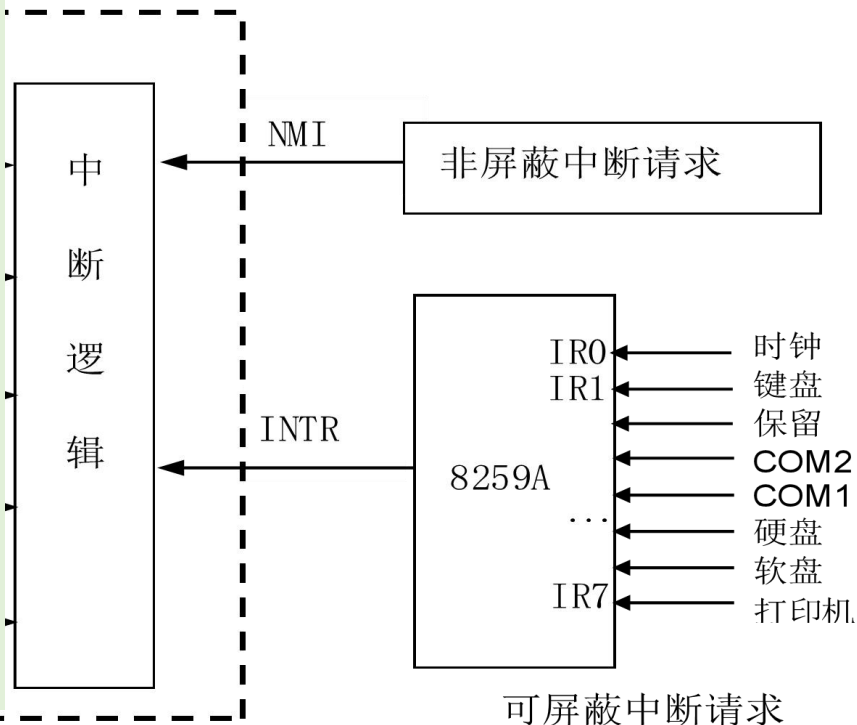


(2) 中断源和8086中断分类

- 发出中断申请的外设或内部原因称为**中断源**
- 中断源进行编号：**中断类型号**，范围**0 ~ 255**，对应256类中断
- 8086中断源有两种：
 - 外部中断（硬件中断），由引脚NMI和INTR引入
 - 内部中断（软件中断），CPU运行过程中发生的一些异常，或者由中断指令INT n引起

■ NMI: 非屏蔽中断

- 比较重要的事件，不能被屏蔽，CPU必须响应，如掉电、存储器错误等等。
- 边沿触发，上升沿之后需要维持两个时钟周期的高电平



■ INTR: 可屏蔽中断

- 受IF标志位控制，IF为1时允许中断，为0时被屏蔽。
- 电平触发，高电平有效，必须维持到中断结束

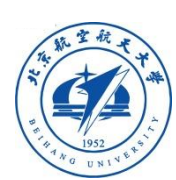
外部中断

中断源	中断类型号
NMI	2
INTR	由8259A提供



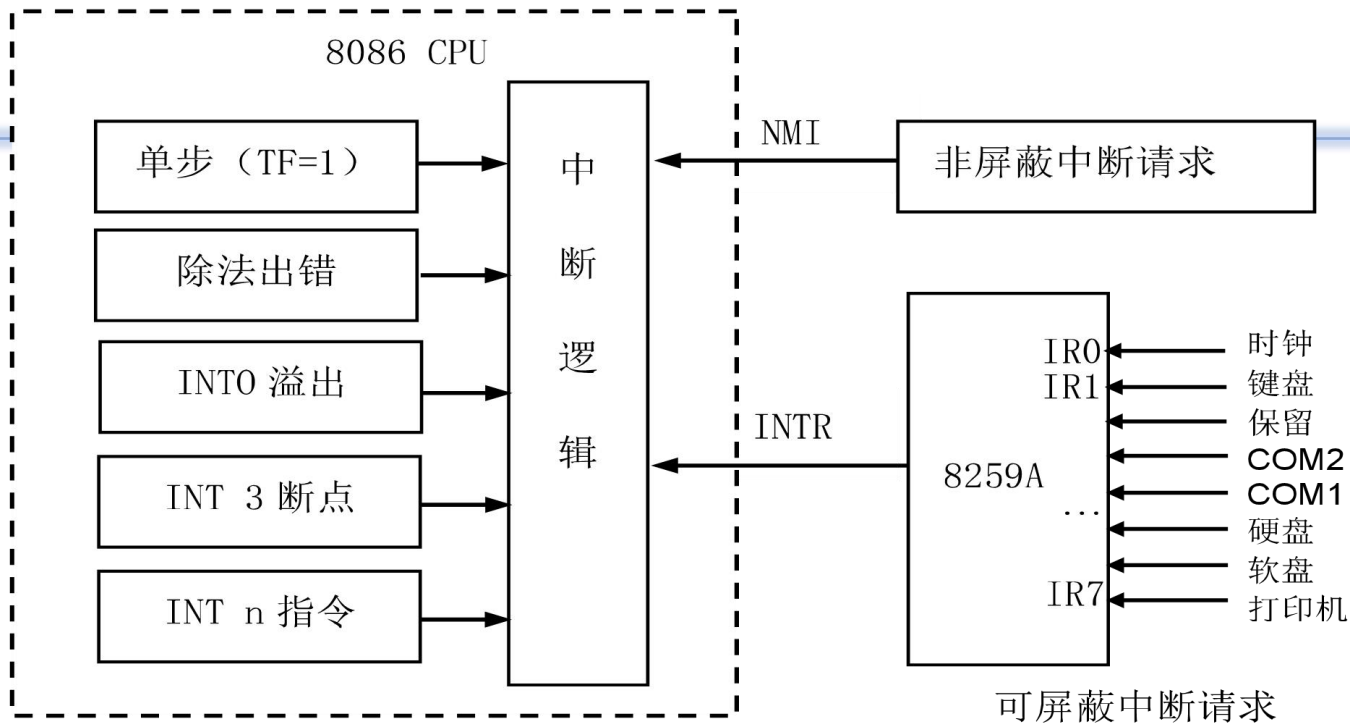
例：PC/XT中的硬件中断源和中断类型号

8259输入	中断类型号	中断源
IRQ0	08	电子钟时间基准
IRQ1	09	键盘
IRQ2	0A	为用户保留
IRQ3	0B	异步通信： COM2
IRQ4	0C	异步通信： COM1
IRQ5	0D	硬盘
IRQ6	0E	软盘
IRQ7	0F	并行打印机



例：PC/AT中的中断源和中断类型号

主 8259A	中断类型号	中断源	从 8259A	中断类型号	中断源
IRQ0	08	日时钟	IRQ8	70	实时时钟
IRQ1	09	键盘	IRQ9	71	改向 INTOAH
IRQ2	0A	从 8259 的 INT	IRQ10	72	保留
IRQ3	0B	串口 2	IRQ11	73	保留
IRQ4	0C	串口 1	IRQ12	74	保留
IRQ5	0D	并口	IRQ13	75	协处理器
IRQ6	0E	软盘	IRQ14	76	软盘
IRQ7	0F	并口 1	IRQ15	77	保留

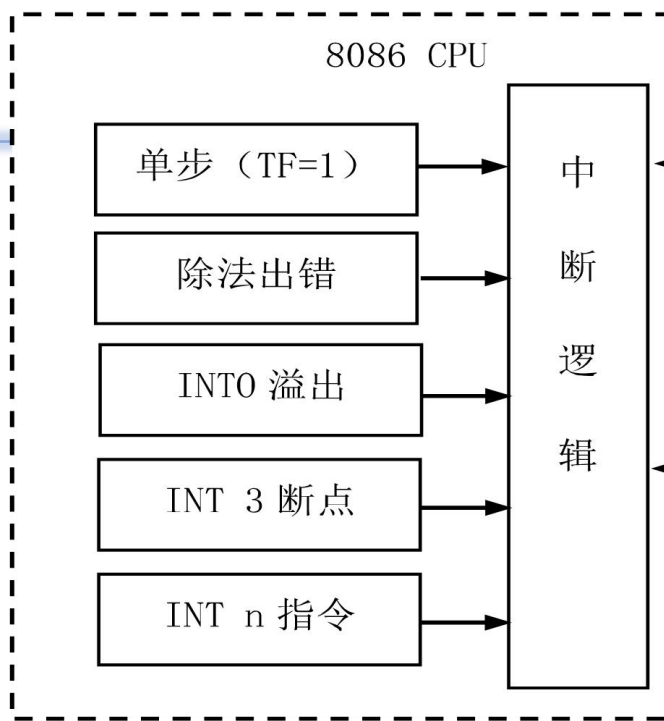


内部中断

中断源	中断类型号
除法错	0
单步中断	1
断点中断	3
溢出中断	4
INT n	n (0~255)

外部中断

中断源	中断类型号
NMI	2
INTR	由8259A提供



①除法错中断（类型号0）

➤ 除法运算时，如果除数为0或商超出了寄存器表达范围，则CPU产生除法错中断



②单步中断（类型号1）

➤ 当TF=1时，CPU每执行一条指令就产生一个单步中断

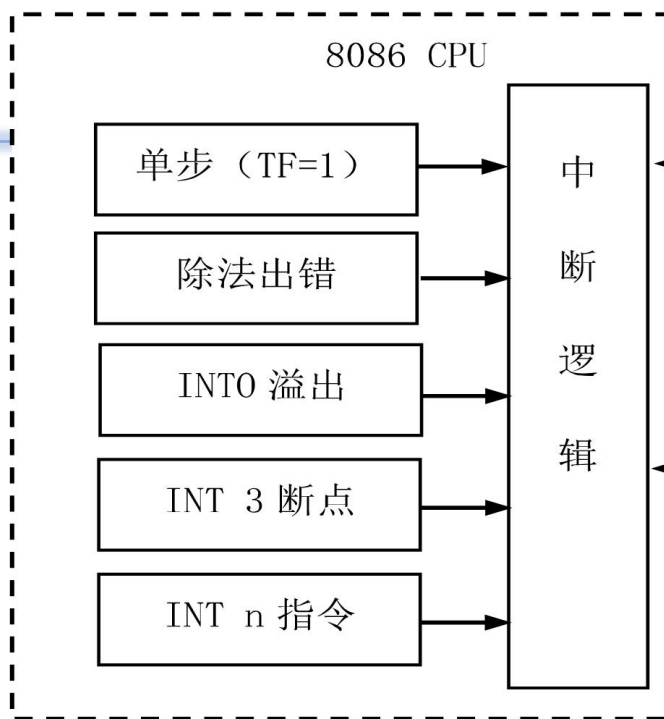
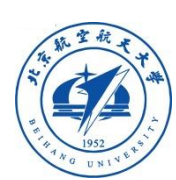
➤ 响应单步中断时会将PSW入栈，然后清除TF和IF，中断处理结束后PSW从堆栈中弹出，CPU被重新置成单步方式

③溢出中断INTO（类型号4）

➤ 当带符号数进行算术运算时，如果OF=1，则可用指令INTO进行处理。如果OF=0，则INTO不起作用

中断源	中断类型号
除法错	0
单步中断	1
断点中断	3
溢出中断	4
INT n	n (0~255)

内部中断



④ **INT n** 软件中断指令(类型号n)

- **n**为中断类型号，其值必须在**0~255**之间
- 可以调用所有**256**类型中断的服务程序
- 可以安排在程序中的任何位置，也称为陷阱中断

中断源	中断类型号
除法错	0
单步中断	1
断点中断	3
溢出中断	4
INT n	n (0~255)

⑤ **断点中断**（类型号3）

- 调试程序时设置断点，实际上是在断点位置安插了一条**INT 3指令**。CPU执行到这个位置就产生类型**3**中断

内部中断

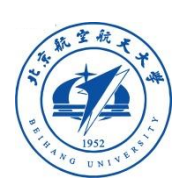


4) 中断类型号的获取

中断源	中断类型号及来源
除法错	0, CPU
单步中断	1, CPU
NMI	2, CPU
断点中断	3, CPU
溢出中断	4, CPU

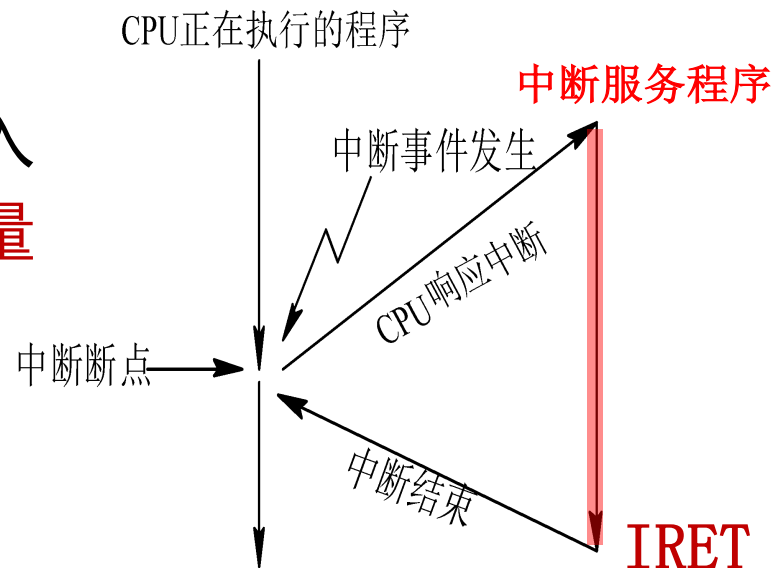
中断源	中断类型号及来源
INT n	n (0~255), 指令
INTR	x (5~255), 硬件电路

- 除法错、单步中断、不可屏蔽中断NMI、断点中断、溢出中断，CPU分别自动提供中断类型号0 ~ 4
- 中断指令INT n，类型号由n决定
- 外部可屏蔽中断INTR，由硬件电路（如8259A）提供中断类型号



3.中断服务程序和中断向量表

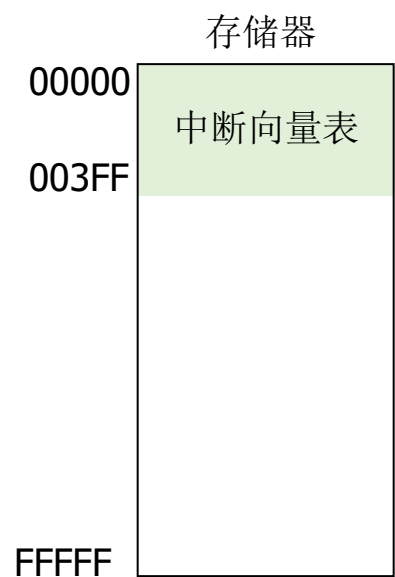
- 处理中断请求，完成其所要求功能的程序，称**中断服务程序**（中断例行程序、中断子程）
- 中断服务程序起始地址（或称为入口地址）称为**中断向量**或**中断矢量**（Interrupt Vector）
- 在中断服务程序中执行指令**IRET**，可以返回到断点



■每个中断向量包括段基址CS、偏移地址IP，共4个字节

■将256类中断的中断向量存储在内存00000 ~ 003FFH的区域，称为中断向量表或中断矢量表

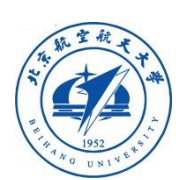
- 中断向量存放地址 = 中断类型号 $n * 4$
- 偏移地址IP存在低地址
 - 段基址CS存在高地址



中断向量表

专用中断	000	类型0中断入口(除法出错)	IP
	004	类型1中断入口(单步中断)	CS
	008	类型2中断入口(NMI)	IP
	00C	类型3中断入口(断点中断)	CS
	010	类型4中断入口(溢出中断)	
系统备用	014	类型5中断入口	
	⋮	⋮	
	07F	类型31中断入口	
供用户使用	080	类型32中断入口	
	⋮	⋮	
	3FC	类型255中断入口	IP
	3FFH		CS

7 0



中断向量存放地址 = 中断类型号 n * 4

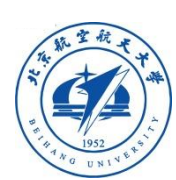
- 例8.1 类型号 $n=44$ 的中断服务程序入口地址为 $3600:2000H$ ，则其存放位置？

存放的位置(物理地址): $n*4 = 44*4 = 176 = B0H$

- 例8.2 中断向量表中 $(0040H) = 240BH$ ，
 $(0042H) = D169H$ ，则对应的中断类型号 $n = ?$ 中断服务程序的起始地址是多少？

中断类型号 $n = 0040H/4 = 10H$

中断服务程序的起始地址 $D169:2408H$

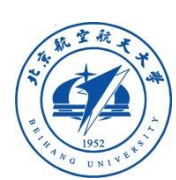


例：执行指令INT 68H， $(1A0H)=2050H$ ， $(1A2H)=A000H$ 。则中断响应过程

- ① 取中断类型号68H
- ② 计算中断向量地址 $68H \times 4 = 1A0H$
- ③ 取中断入口地址：
 - $IP = (1A0H) = 2050H$
 - $CS = (1A2H) = A000H$
- ④ 执行中断服务程序
- ⑤ 中断处理结束，执行IRET指令返回到断点(INT 68H指令的下一条指令)

□ 步骤③之前，CPU自动将PSW、CS、IP依次入栈

□ 步骤⑤执行IRET时会将栈顶的数据依次弹出到IP、CS、PSW



4.中断优先级和中断嵌套

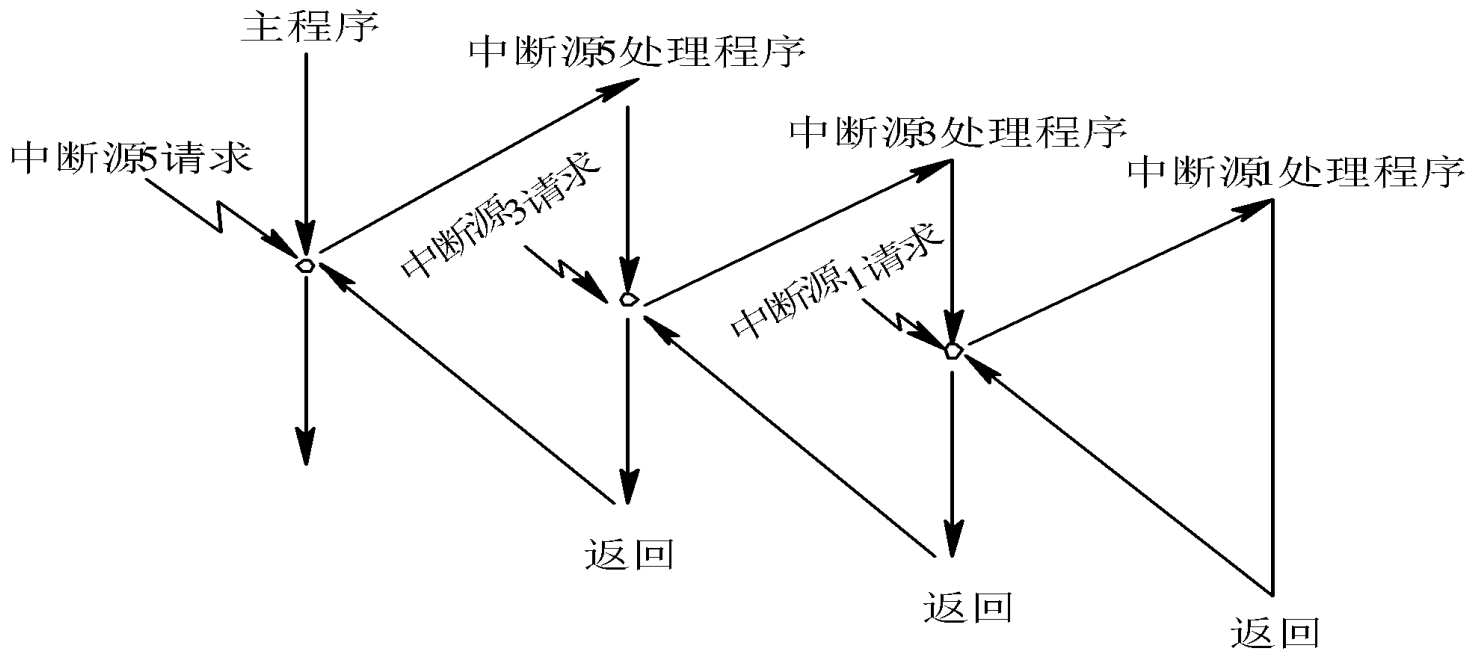
- 当多个中断源同时提出中断申请时，需要按照中断源的重要性和实时性等因素，为它们排出响应的次序。这个响应次序称为**中断优先级**

8086中断优先级

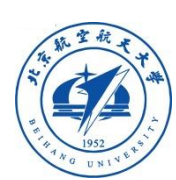
中断源	优先级
除法错、INT n，INT O	最高
NMI	↑
INTR	
单步（陷阱）	最低



- 通常情况下，如果CPU正在处理中断时，有更高优先级的中断到来，而且此中断没有被屏蔽，则CPU能够响应此更高优先级的中断，这就形成了**中断嵌套**



(假设中断源1~n的优先级为从高到低)



8.1.2 中断响应与处理过程

- 当中断源产生中断申请后，只要满足**响应条件**，在**执行完当前指令后**，CPU内部硬件会**自动完成响应中断的过程**



1) 响应条件

CPU在执行完当前指令后才能响应中断。

■ 响应INTR中断请求，还需要满足以下条件：

- 8086的PSW中**IF=1**（中断允许）
- 在8259A中未被屏蔽

■ NMI请求不受IF标志位影响

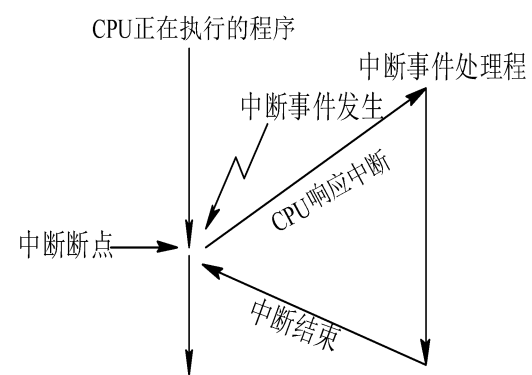
- 如果在处理NMI中断的过程中，NMI引脚又有中断请求，则CPU仍然能再次响应。

■ 软件中断INT n由程序设定，不受IF位影响。在执行INT n中断服务程序过程中

- 如果有NMI请求，则在执行完当前指令后响应中断；
- 如果有INTR请求而且IF=1，则在执行完当前指令后响应中断。



2) 中断响应过程



① 取中断类型号N

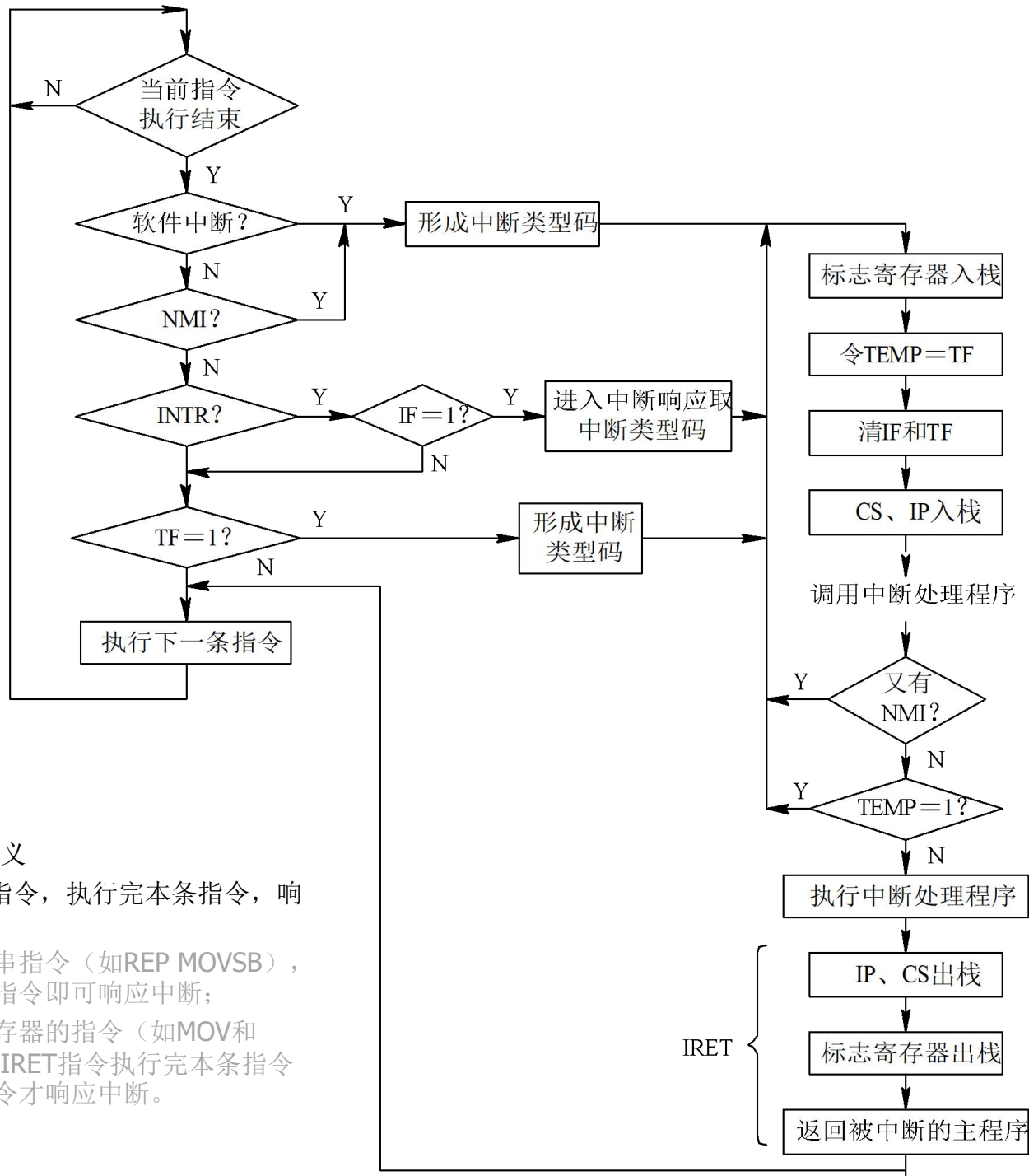
当前PSW、CS、IP内容入栈;

然后PSW中的IF、TF标志清0

② 取内存单元($N \times 4$)字内容送IP

取内存单元($N \times 4 + 2$)字内容送CS

③ 中断处理程序结束后, 执行IRET指令, 从堆栈依次弹出数据到IP、CS、PSW, 返回断点处继续执行。



当前指令执结束的含义

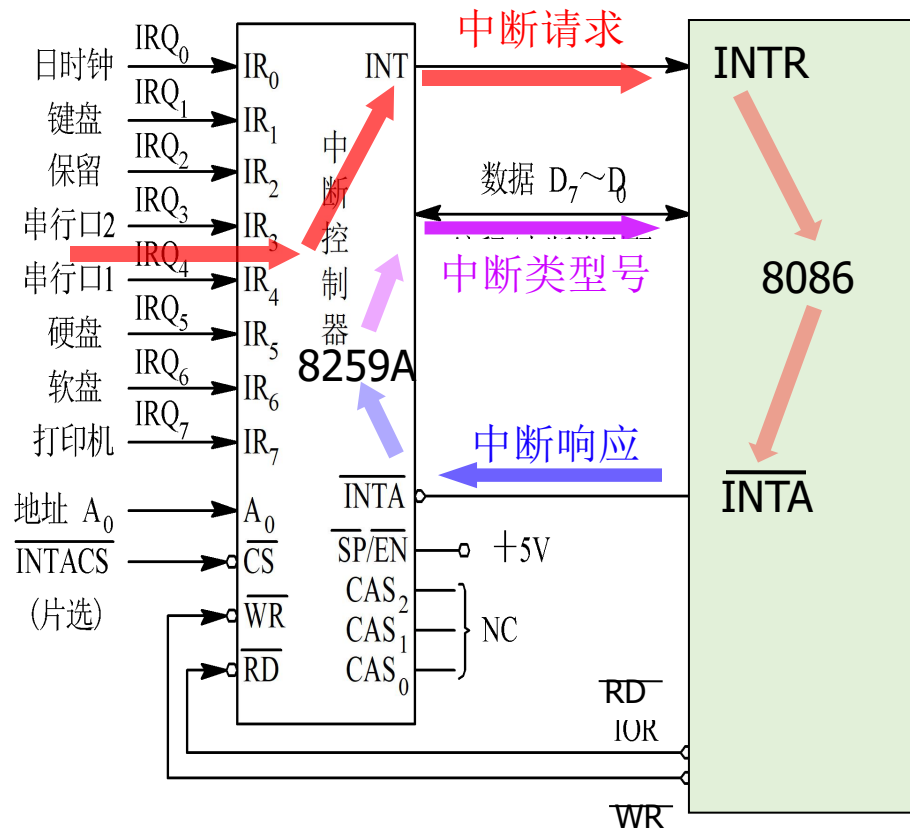
- CPU正在执行一条指令，执行完本条指令，响应中断；
- 对于带重复前缀的串指令（如REP MOVSB），执行一次重复和串指令即可响应中断；
- 对处理对象为段寄存器的指令（如MOV和POP），以及STI和IRET指令执行完本条指令后，再执行一条指令才响应中断。

IRET



3) INTR中断处理

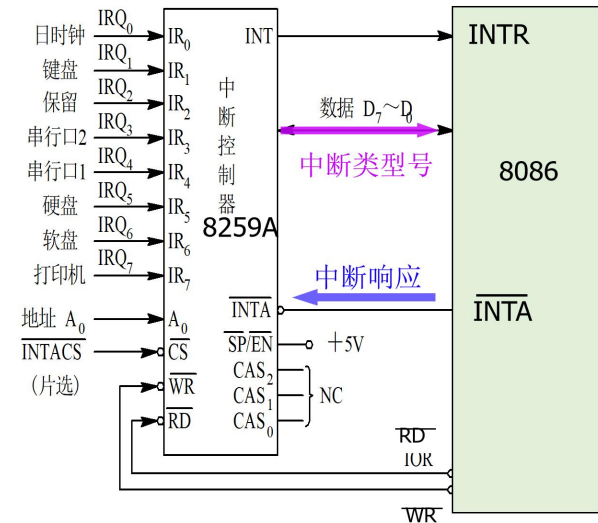
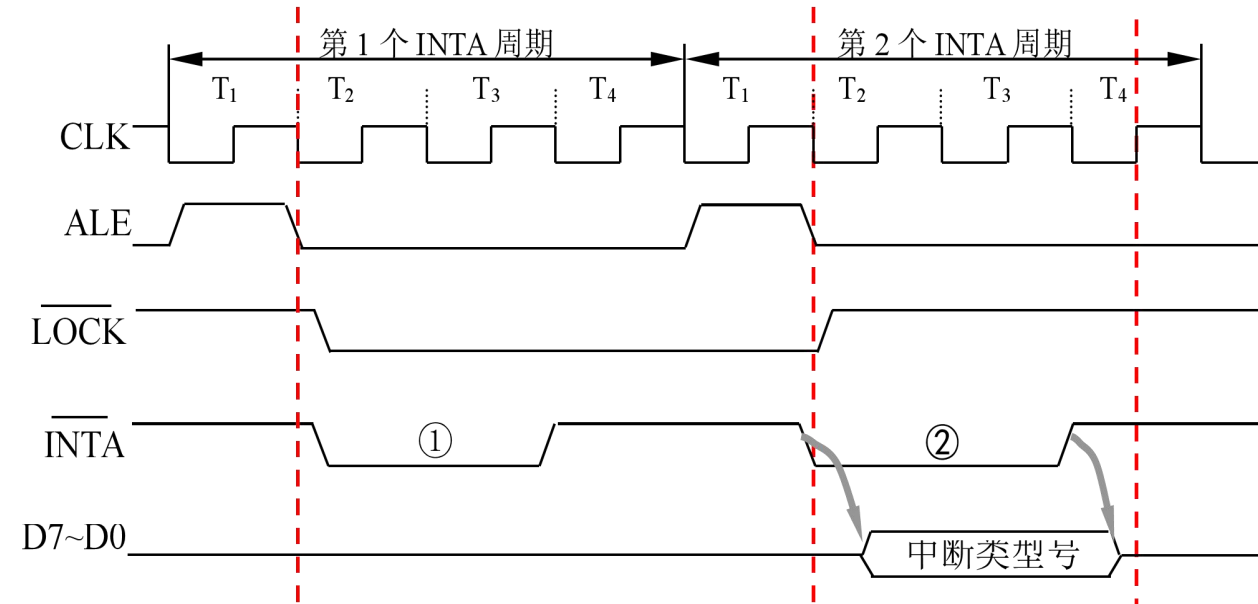
- ① 外设产生断信号，在8259A中经过优先级判断，向CPU提出中断请求INTR
- ② CPU响应中断，获取中断类型号（8259A提供）
- ③ CPU接收到中断类型号做出处理，转向中断服务程序
- ④ 中断服务程序返回。中断响应结束。





3) INTR中断处理

- 8086响应INTR的时序：需用**两个连续的 $\overline{INTA}$ 总线周期**，每个总线周期包括四个时钟周期(T1~T4)



- 第一个 $\overline{INTA}$ 周期，CPU在T2~T4期间发出中断响应信号 $\overline{INTA}$ 给8259A；
- 第二个 $\overline{INTA}$ 周期中，CPU发出第二个 $\overline{INTA}$ 信号。8259A响应第二个 $\overline{INTA}$ ，在T2~T3将中断类型号N送到数据总线。
- (CPU得到中断向量后，PSW入栈，清IF和TF位，断点返回地址CS:IP入栈，然后转向中断服务程序)

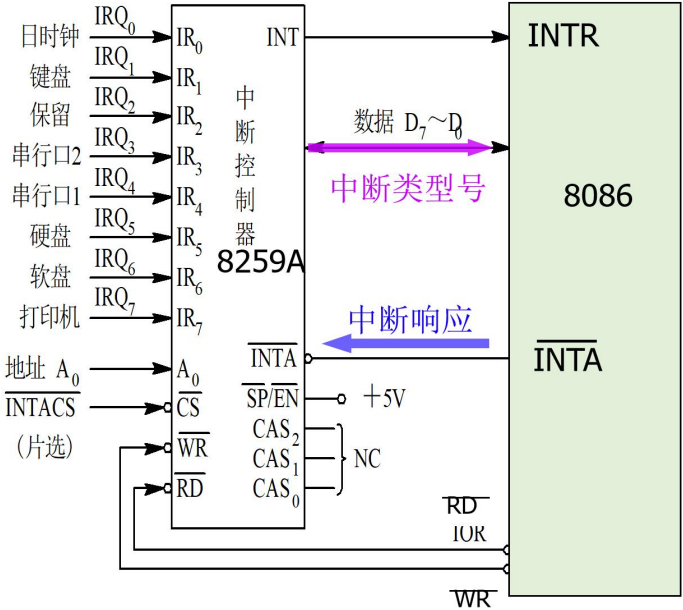
INTR中断类型号的确定(了解)

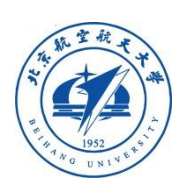
中断类型号高5位：
8259A初始化时由
程序设置

中断类型号低3位：
产生中断时由
8259A自动生成

8259A引入的INTR中断类型号的确定

D7-D3	D2-D0	中断类型号	中断输入引脚	中断源
00001	000	08H	IR0	系统时钟
00001	001	09H	IR1	键盘
00001	010	0AH	IR2	保留
00001	011	0BH	IR3	COM2
00001	100	0CH	IR4	COM1
00001	101	0DH	IR5	硬盘
00001	110	0EH	IR6	软盘
00001	111	0FH	IR7	打印机





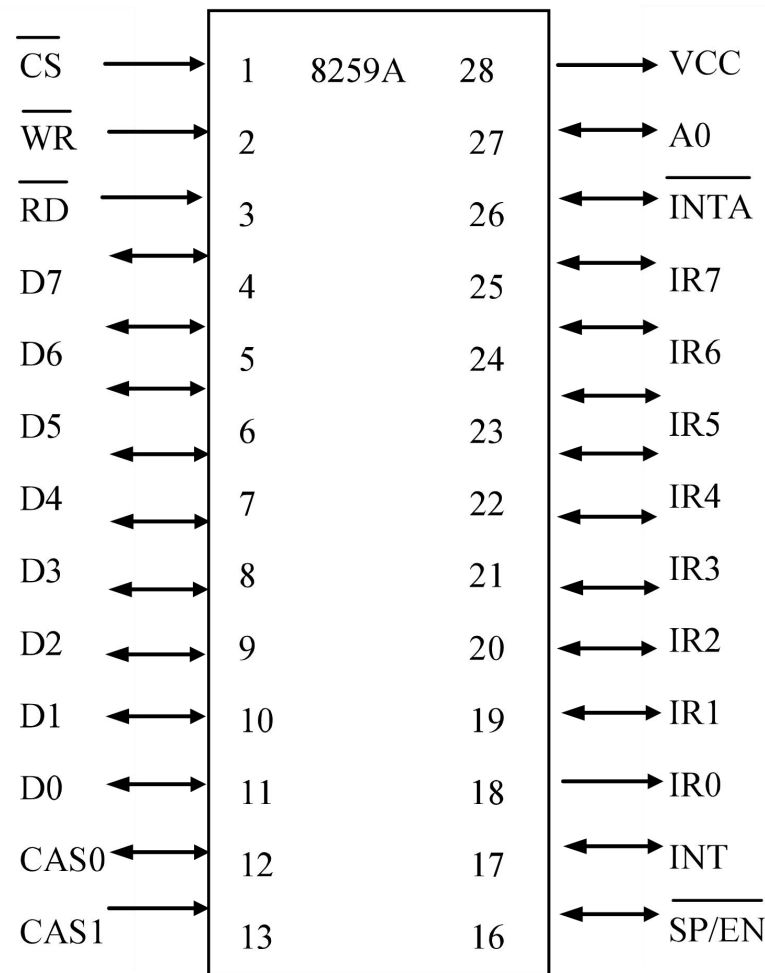
8.2 8259A简介(了解)

- 引脚与级联
- 内部结构
- 对INTA响应原理
- 工作方式

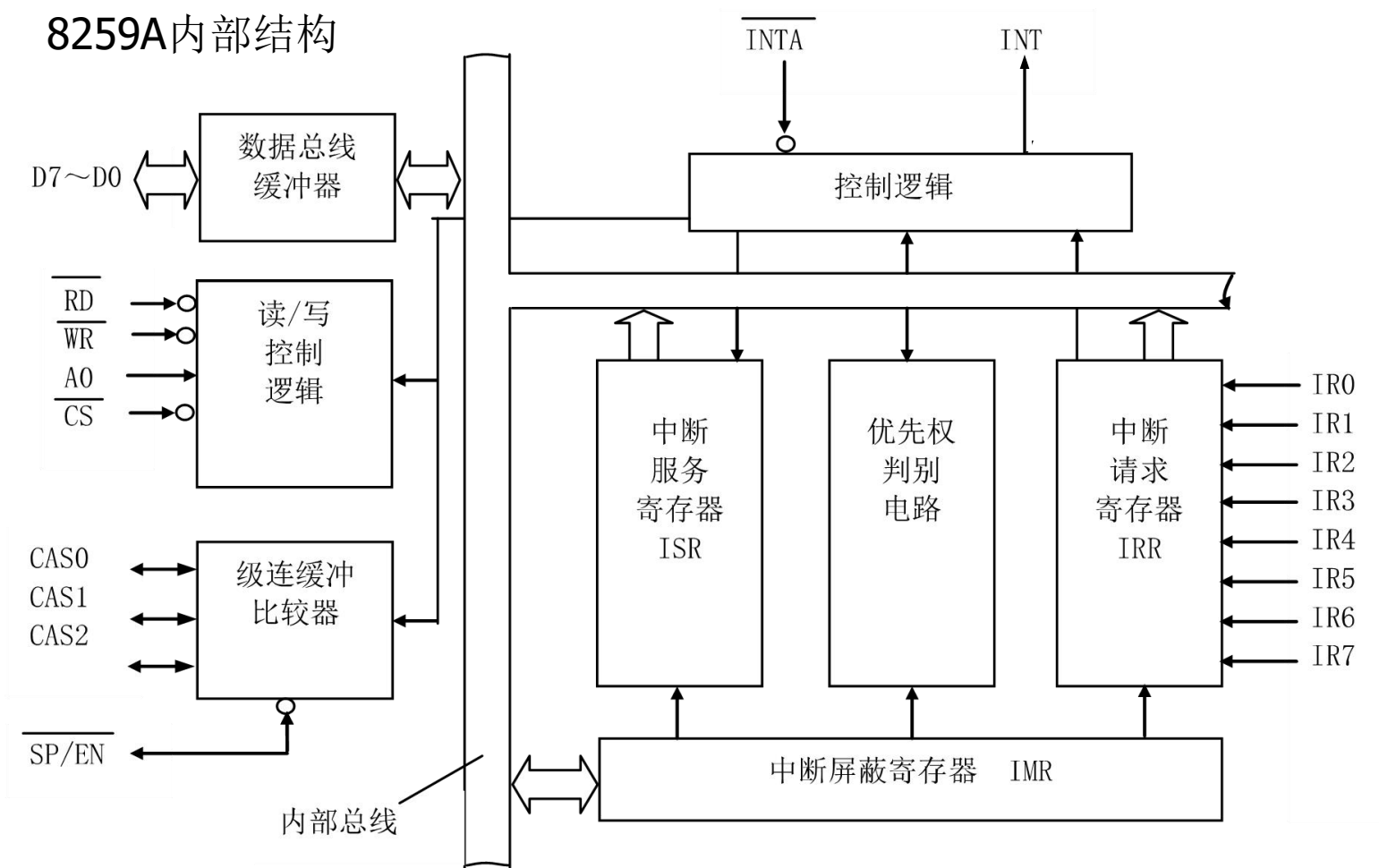


8.2.1 8059A引脚和内部结构

- IR0~IR7：接外设中断请求信号
- INT：发送给CPU的中断请求信号
- $\overline{\text{INTA}}$ ：接收CPU的中断响应信号
- D0~D7：数据引脚
- $\overline{\text{CS}}, \overline{\text{RD}}, \overline{\text{WR}}$ ：控制引脚
- A0：端口选择引脚
- CAS0~CAS2：用于级联
- SP/EN



8259A内部结构

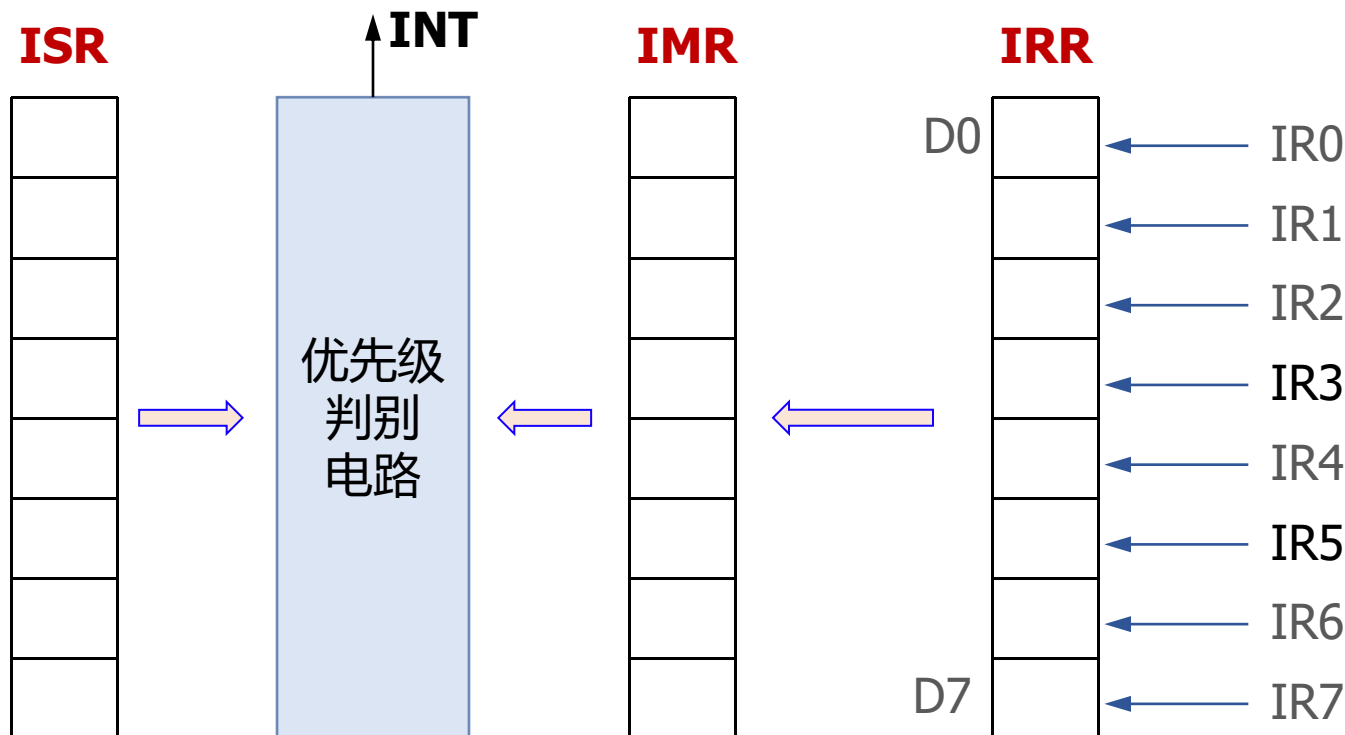


1. 中断请求寄存器 IRR
2. 中断服务寄存器 ISR
3. 中断屏蔽寄存器 IMR

4. 中断优先级判断器
5. 控制逻辑 (ICW_{1~4}, OCW_{1~3})
6. 读/写控制逻辑
7. 级连缓冲器/比较器

1. 8059A中断优先级判别与中断请求(INT)信号的形成

□ 8259A工作在普通全嵌套方式或特殊全嵌套方式, 则优先级顺序: $IR0 > IR1 > \dots > IR7$



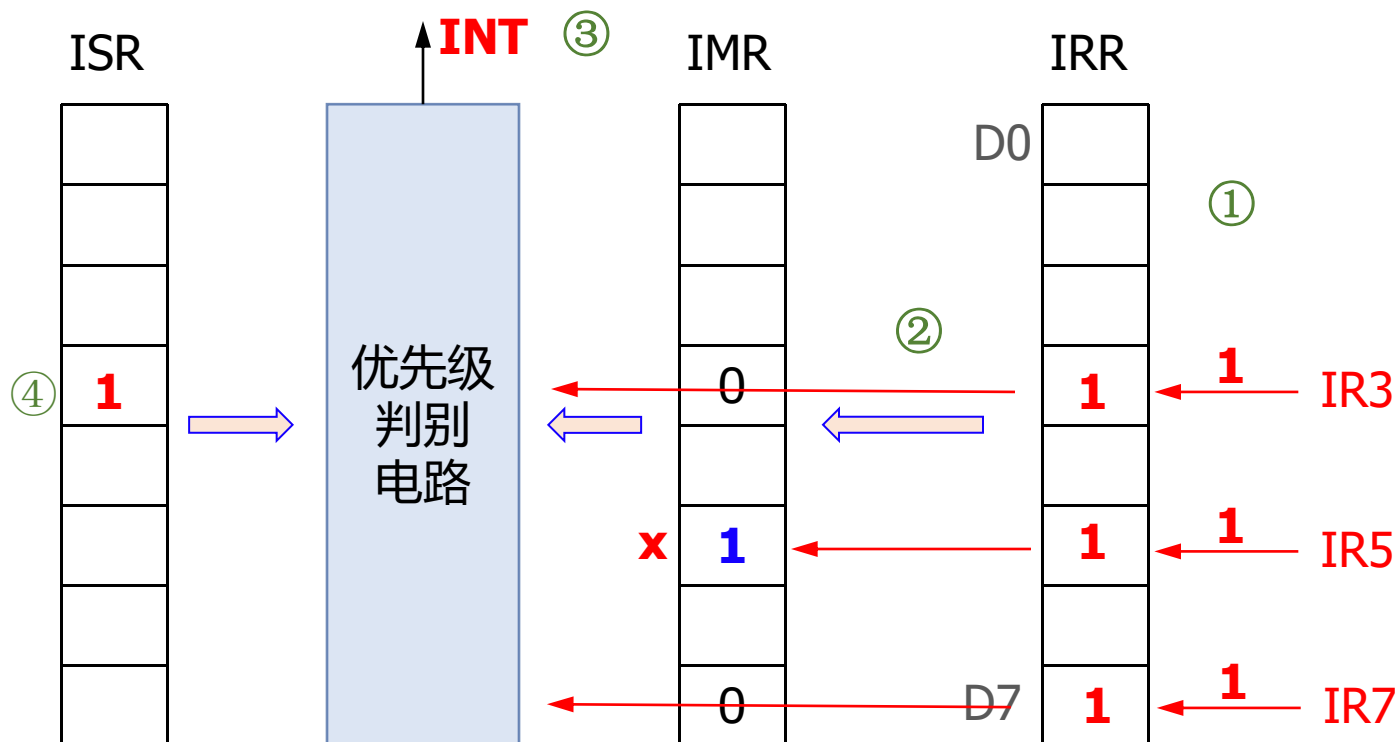
1. **IRR:** 中断请求寄存器。当外设有中断请求时, 相应位**置1**
2. **IMR:** 中断屏蔽寄存器。某一位**为1**时, 则相应的外设中断**被屏蔽**
3. **ISR:** 中断服务寄存器。8086响应某个外设的中断, 则相应的位**置1**

1. 8059A中断优先级判别与中断请求(INT)信号的形成

□ 8259A工作在
普通全嵌套方式或
特殊全嵌套方式,

则优先级顺序:

IR0>IR1>...>IR7



①如图所示，IR3、IR5、IR7有中断请求信号，因此相应的位被置1

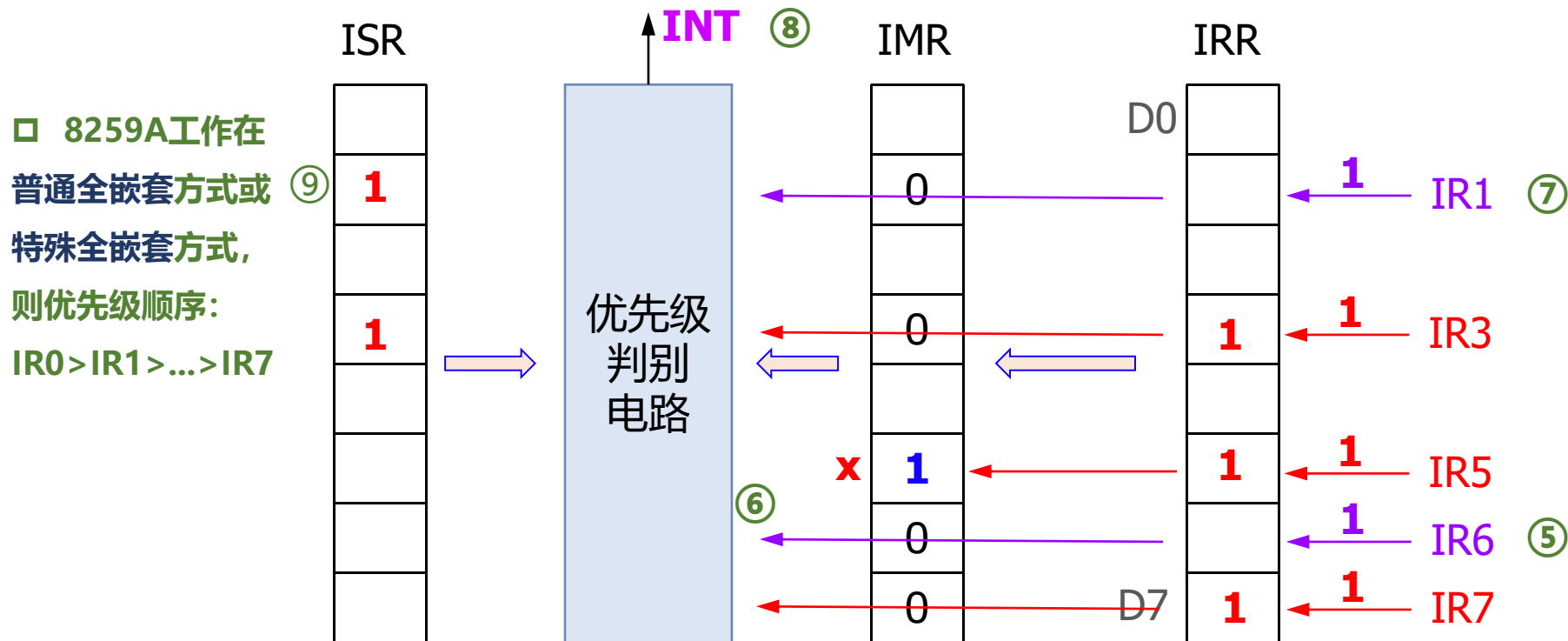
② a) IMR.D3=0，IMR.D7=0，因此IR3、IR7的中断请求信号没有被屏蔽

② b) IMR.D5=1，因此IR5的中断请求信号被屏蔽，不能做进一步处理

③ ISR各个位均为0，因此优先级判别电路仅比较IR3和IR7的优先级，并在INT引脚输出IR3中断请求信号

④ CPU响应INT信号，则ISR.D3被置1，表示CPU正在处理IR3的中断。IR7则被挂起。等IR3中断处理结束、ISR.D3清零之后才能响应。

1. 8059A中断优先级判别与中断请求(INT)信号的形成



- ⑤如果IR3尚未处理结束, IR6又产生了中断请求, 且IMR中没有屏蔽。则IR6的中断请求被送至优先级判别电路。
- ⑥由于此时ISR.D3=1, 对应的IR3优先级更高, 因此8259A不会向CPU发出中断请求INT。IR6的中断请求被挂起。
- ⑦接着IR2产生中断请求, 且IMR中没有屏蔽。则其中断请求被送至优先级判别电路。
- ⑧IR2比IR3优先级高, 因此8259A会再次发出INT信号
- ⑨CPU响应INT信号, 处理IR2中断。ISR.D2被置1



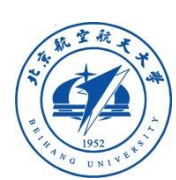
2. 8259A对INTA的响应

第一个 $\overline{\text{INTA}}$ 负脉冲到达时，8259A进行以下处理：

- 1) 使IRR的锁存功能失效。
- 2) 使当前中断服务寄存器ISR中的相应位置1。
- 3) 使IRR寄存器中的相应位清零。

第二个 $\overline{\text{INTA}}$ 负脉冲到达时，8259A进行以下处理：

- 1) 将中断类型寄存器中的内容ICW2，送到数据总线的D7~D0，CPU将此作为中断类型码。
- 2) 第二个脉冲结束时，8259A会将第一个脉冲到来时设置的当前中断服务寄存器ISR的相应位清零（自动EOI方式）。



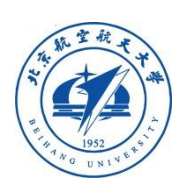
8.2.2 8259A工作方式--优先级方式

1) 全嵌套方式

- 默认方式。引脚IR0→IR7的中断优先级依次降低
- 中断处理过程中允许高级中断打断低级中断，禁止同级或低级中断进入
- 中断结束时，CPU发送EOI命令（实际上是某种格式的控制字）清除ISR位。或者AEOI模式自动清除。

2) 特殊全嵌套方式

- 与全嵌套类似的区别：允许同级中断进入。



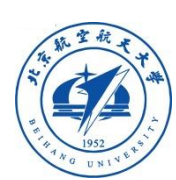
8.2.2 8259A工作方式--优先级方式

3) 优先级自动循环方式

- 初始状态下优先级顺序从高到低 $IR_0 \rightarrow IR_7$
- 当 IR_i 的中断服务结束后, 优先级自动降为最低,
 IR_{i+1} 优先级变为最高, 其它引脚优先级顺延。

4) 优先级特殊循环方式

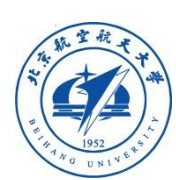
- 与自动循环优先级的区别: 初始化的时候预先设置
最低优先级引脚。



8.2.2 8259A工作方式--中断屏蔽方式

可以使用CLI指令将8086 PSW中IF位清零，屏蔽INTR中断。除此之外，还可以使用8259A中的IMR寄存器屏蔽对应引脚的中断。

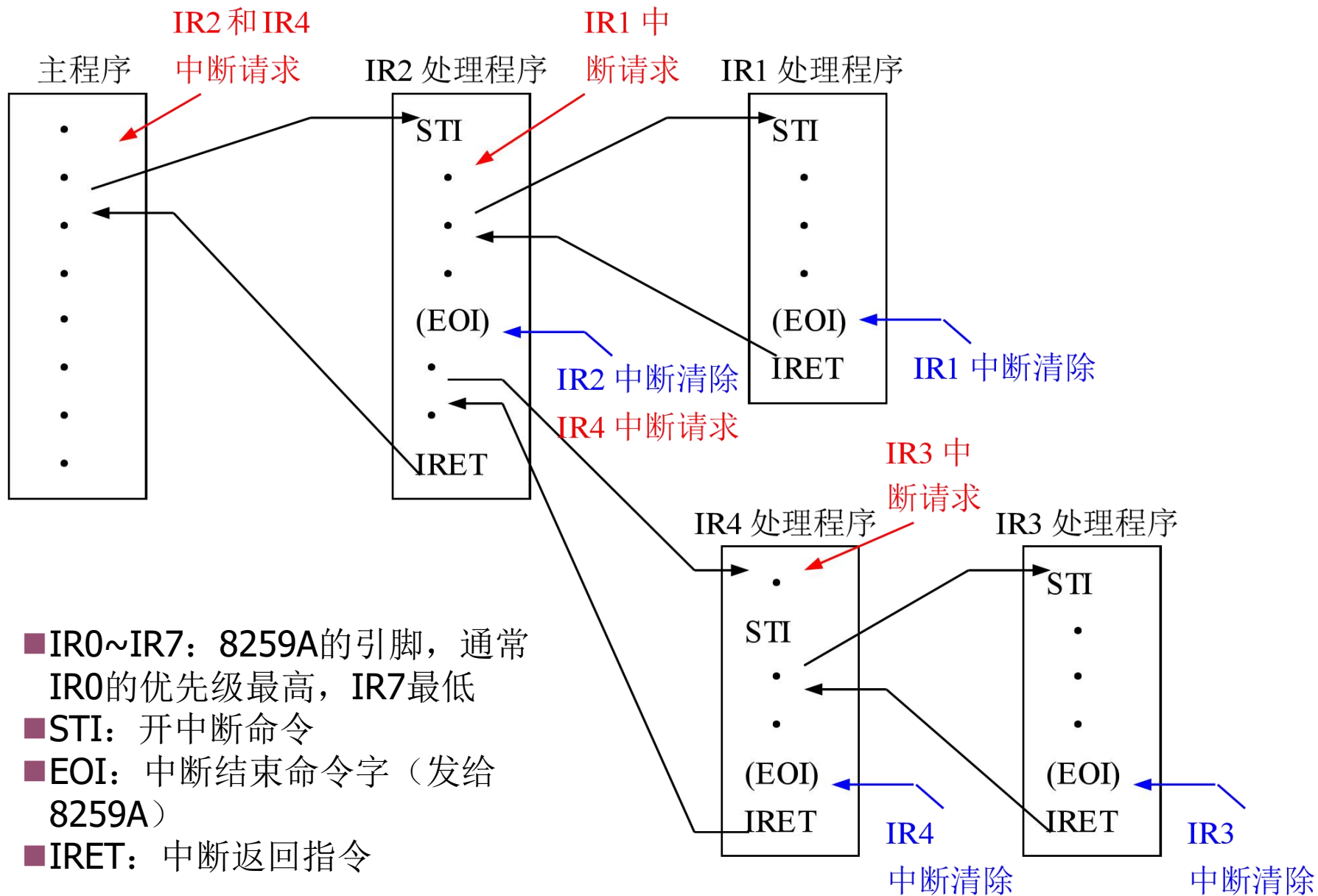
- 1) 普通屏蔽方式：IMR中某些位置1，屏蔽相应的引脚。
- 2) 特殊屏蔽方式：处理某中断的过程中，仅仅屏蔽同级中断自身，而允许高级和低级中断进入。



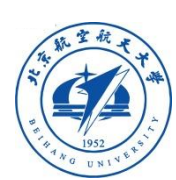
8.2.2 8259A工作方式--结束中断方式

中断请求被CPU响应后，ISR寄存器相应位 IS_n 置1。中断处理结束后应当将该位清0。对 IS_n 清0的操作称为结束中断。

- 自动中断结束方式 (AEOI) : CPU发出第二个INTA信号时自动清除 IS_n 位。
- 普通中断结束方式 (EOI) : CPU处理完中断请求时，向8259A发送一个EOI命令（实际上是特定格式的控制字）。8259A接受到之后，清除ISR中当前优先级最高的 IS_n 位。
- 特殊中断结束方式 (SEOI) : 在EOI命令中指定对ISR的哪一位清0。



8086中断嵌套及EOI命令的使用



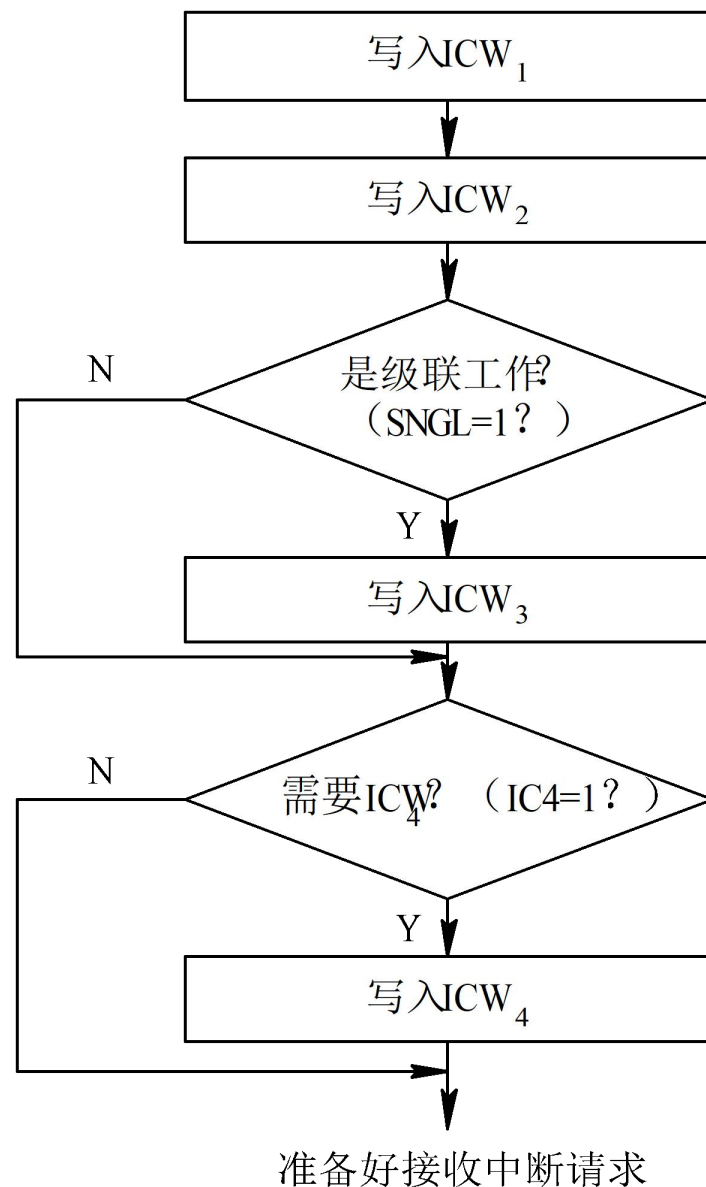
8.2.3 8259A命令字及编程

- 上节所述各种工作方式全靠8259A内部两类控制寄存器控制：
 - 初始化控制字：ICW1~ICW4
 - 操作控制字：OCW1~OCW3

1)初始化控制字的写入

初始化控制字的写入有严格的顺序。

- 首先写ICW1，然后是ICW2
- ICW1中SNGL位为1，则写ICW3，否则不写
- ICW1中IC4位为1，则写ICW4，否则不写

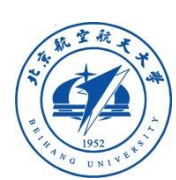




2)操作控制字的写入

- 操作控制字的写入顺序没有要求

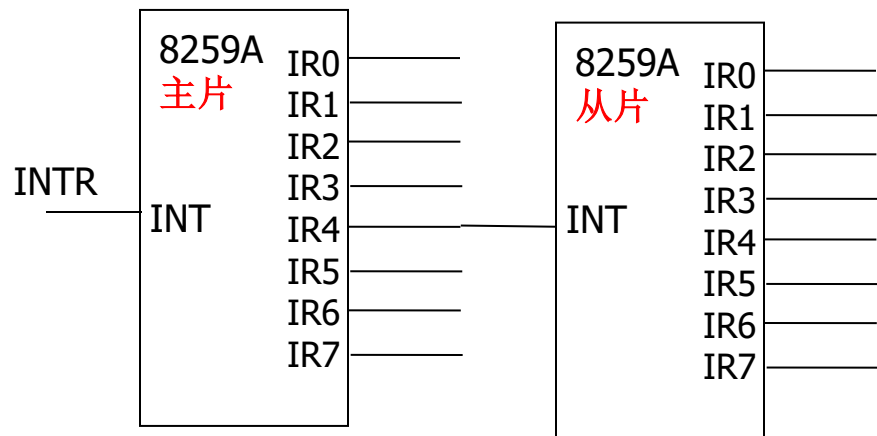
3) 8259A命令字及其编程（略）



8.3 8259A级联

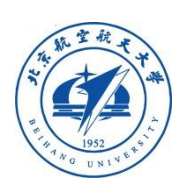
多个8259A可以进行级联，以扩展计算机系统的中断源数量。

- 级联方式：8259A从片INT引脚，连接8259A主片的IR0~7引脚
- 主片最多连接8个从片，因此中断源数量最多 $8*8=64$ 个。
- 主片工作在特殊全嵌套方式，从片工作在普通全嵌套方式。



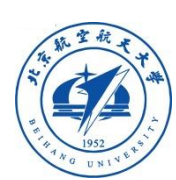
➤ 图中各个引脚的优先级

主片($IR0 > IR1 > IR2 > IR3$) > 从片($IR0 > IR1 > \dots > IR7$) > 主片($IR5 > IR6 > IR7$)



8.4 8086中断服务程序设计

- 用户可以编写自己的中断服务程序，加载到系统中，并在中断向量表中设置该程序的入口地址（中断向量）
 - 中断向量的设置
 - 中断服务子程序设计（略）



1. 中断向量设置

- 用户自己定义的中断类型号，可以用INT n来引用，也可以通过8259A/INTR触发中断执行
- 不允许用户修改的中断向量
 - 专用中断，类型号0 ~ 4;
 - 系统使用中断，类型号5 ~ 3FH（包括中断21H）；
 - 8259A中断，类型号08H~0FH;
 - BIOS中断，类型号10H~1FH

1) 用指令直接设置中断向量

- 例8.15 用指令设置中断服务程序入口地址到中断类型号n对应的中断向量表中。

```
MOV  AX, 0
```

```
MOV  ES, AX
```

```
;写中断向量偏移地址
```

```
LEA  AX, INTRAD
```

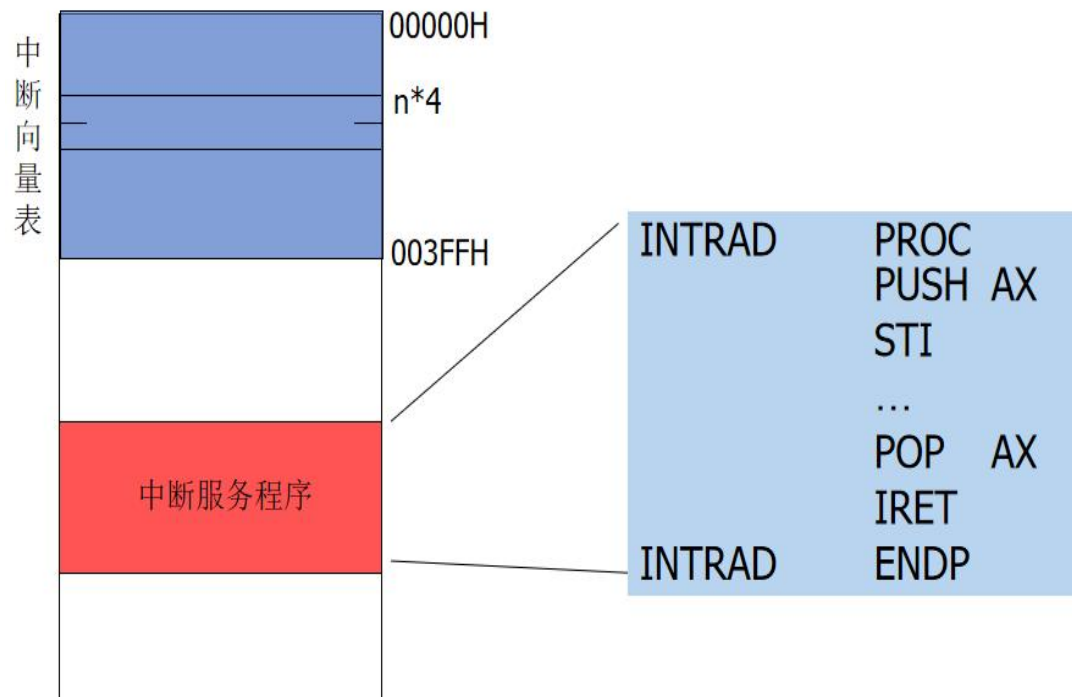
```
MOV  BX, N*4
```

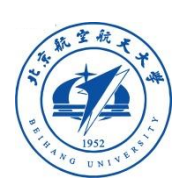
```
MOV  ES:[BX], AX
```

```
;写中断向量段基址
```

```
MOV  AX, SEG INTRAD
```

```
MOV  ES:[BX+2], AX
```





2) 用DOS功能调用设置

(1) 设置中断向量

- 入口参数:

AL = 中断类型号N

DS:DX = 中断向量

AH = **25H**

- 执行: INT 21H

- 结果: 将中断服务程序入口地址放入中断向量表中
类型号N对应的位置

(2) 取中断向量

- 入口参数:

AL = 中断类型号N

AH = **35H**

- 执行: INT 21H

- 返回: ES:BX = 中断向量

例8.16 利用DOS功能调用设置中断向量

;中断向量段基址->DS

```
MOV    AX, SEG INTRAD
```

```
MOV    DS, AX
```

;中断向量偏移地址->DX

```
MOV    DX, OFFSET INTRAD
```

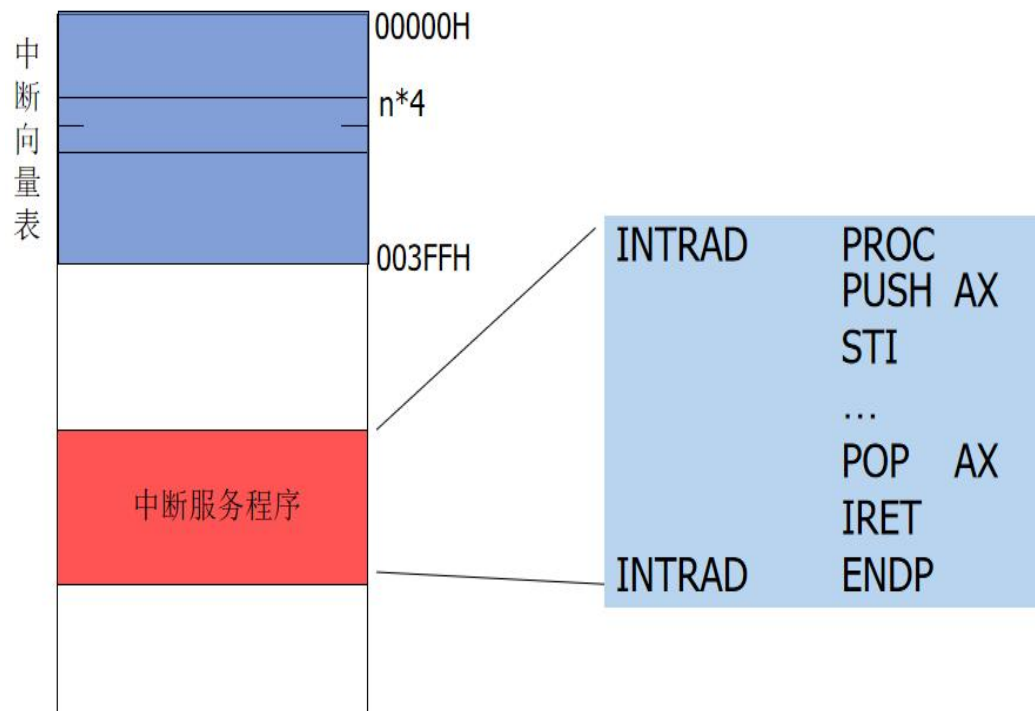
;中断类型号->AL

```
MOV    AL, N
```

;设置中断向量

```
MOV    AH, 25H
```

```
INT    21H
```



例8.16-1 利用DOS功能调用取中断向量和设置中断向量

1. 获取原有中断向量并保存
2. 设置新中断向量
3. 程序结束后恢复原来的中断向量

;取类型号N的中断向量到ES:DX

```
MOV    AL, N
MOV    AH, 35H
INT     21H
```

;保存原中断向量

```
PUSH    ES
PUSH    BX
```

```
PUSH    DS
```

;中断向量段地址->DS

```
MOV     AX, SEG INTRAD
MOV     DS, AX
```

;中断向量偏移地址->DX

```
MOV     DX, OFFSET INTRAD
```

;中断类型号->AL

```
MOV     AL, N
```

;设置中断向量

```
MOV     AH, 25H
INT     21H
POP     DS
```

...

;程序结束时，恢复中断向量

```
POP     DX
POP     DS
```

```
MOV     AL, N
MOV     AH, 25H
INT     21H
```

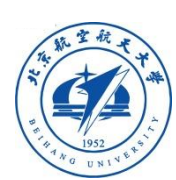
;中断服务子程序

```
INTRAD PROC
```

...

```
IRET
```

```
INTRAD ENDP
```



8.4.2 中断服务子程序（了解）

- 中断服务子程序用于处理中断，完成预先指定的功能

INTRAD PROC ;中断服务子程序框架

```
PUSH AX  
PUSH BX  
PUSH SI
```

①保护中断现场。
根据需要将数据
压入堆栈

②开中断，允许中断嵌套 → STI

```
... ..
```

③处理中断

```
MOV AL, 20H  
OUT 20H, AL
```

④给8259A的中断
结束(EOI)命令。
需要时才发送

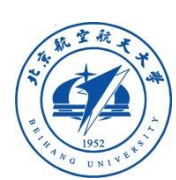
⑤关中断，恢复中断现场时不能被打断 → CLI

```
POP SI  
POP BX  
POP AX
```

⑥恢复中断现场。
弹出顺序与入栈
相反

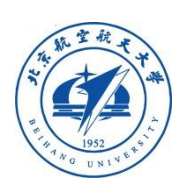
⑦返回主程序 → IRET

INTRAD ENDP



1. 中断服务子程序的一般结构顺序

- ① 程序开始必须**保护中断现场**。一般是用一系列**PUSH**指令将CPU各个寄存器的值压入栈中
- ② 若允许中断嵌套，则用指令**STI开中断**，使得 $IF=1$
- ③ 处理中断处理
- ④ 中断服务程序结束前，用指令**CLI关中断**，使得 $IF=0$ ，禁止响应其它中断
- ⑤ 给中断命令寄存器（8259A）送**中断结束命令EOI**，清除当前正在处理的中断请求标志位。
- ⑥ **恢复中断现场**。一般是用**POP**指令恢复CPU各个寄存器的值
- ⑦ 用中断返回指令**IRET返回主程序**。此时堆栈中保存的断点值和标志值分别装入IP, CS,和PSW



2. 注意事项

- 进入中断服务程序时，CPU自动清除IF和TF位。如果允许中断嵌套则需要人为地开中断。
- 退出中断服务程序前，需要恢复寄存器的内容。为了避免又有中断进入破坏其内容，需要关中断，在中断返回时，原来的PSW弹出，会使得 $IF=1$ 。
- 中断结束命令EOI只有在响应某些来自8259A的INTR时才需要，并且只能在中断结束前发出。

例：编写中断处理程序，要求主程序运行时，每2秒屏幕上显示信息“Timer Interrupt apply”。
(参考教材P278 例8-18)

主程序部分

DATA SEGMENT

COUNT DW 36

MESS DB 'Timer Interrupt apply',0DH,0AH,'\$'

DATA ENDS

STACK SEGMENT STACK
ST1 DB 100 DUP(?)

STACK ENDS

CODE SEGMENT

MAIN PROC FAR

ASSUME CS:CODE, DS:DATA

ASSUME ES:DATA, SS:STACK

PC机定时中断（中断类型号1CH）每秒发生18.2次

START:

```
MOV  AX, STACK
MOV  SS, AX
MOV  SP, 100
MOV  AX, DATA
MOV  DS, AX
MOV  AL, 1CH ;获取原中断向量
MOV  AH, 35H
INT  21H
PUSH ES      ;保存原中断向量
PUSH BX
PUSH DS
MOV  DX, OFFSET RING
MOV  AX, SEG RING
MOV  DS, AX
MOV  AL, 1CH ;设置中断向量
MOV  AH, 25H
INT  21H
POP  DS
```

```
IN    AL, 21H ;设置中断屏蔽位
AND   AL, 0FEH ;使IMR第0位为0
OUT   21H, AL ;仅允许定时中断
STI
```

```
MOV  DI, 0FFFFH ;延时期间,
DELAY:      ;不断产生定
MOV  SI, 0FFFFH ;时中断, 在
DELAY1:     ;屏幕上显示
DEC  SI      ;字符串
```

```
JNZ  DELAY1
DEC  DI
JNZ  DELAY
```

```
POP  DX      ;取原中断向量
```

```
POP  DS
```

```
MOV  AL, 1CH
```

```
MOV  AH, 25H
```

```
INT  21H      ;恢复原中断向量
```

```
RET
```

```
MAIN  ENDP
```

中断服务子程序

```
RING PROC NEAR
    PUSH DS
    PUSH AX
    PUSH CX
    PUSH DX
    MOV AX, DATA
    MOV DS, AX
    STI
    DEC COUNT ;2秒计数
    JNZ EXIT
```

```
    MOV DX, OFFSET MESS
    MOV AH, 09H ;显示信息
    INT 21H
    MOV COUNT, 36

EXIT:  CLI
    POP DX
    POP CX
    POP AX
    POP DS
    IRET

RING ENDP
CODE ENDS
END START
```




例二

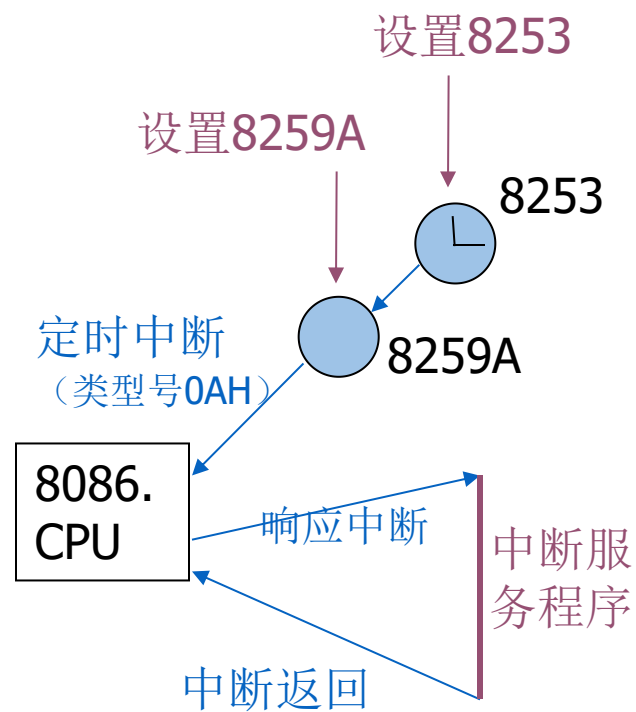
在PC/XT上用ADC0809设计一块8通道的数据采集卡

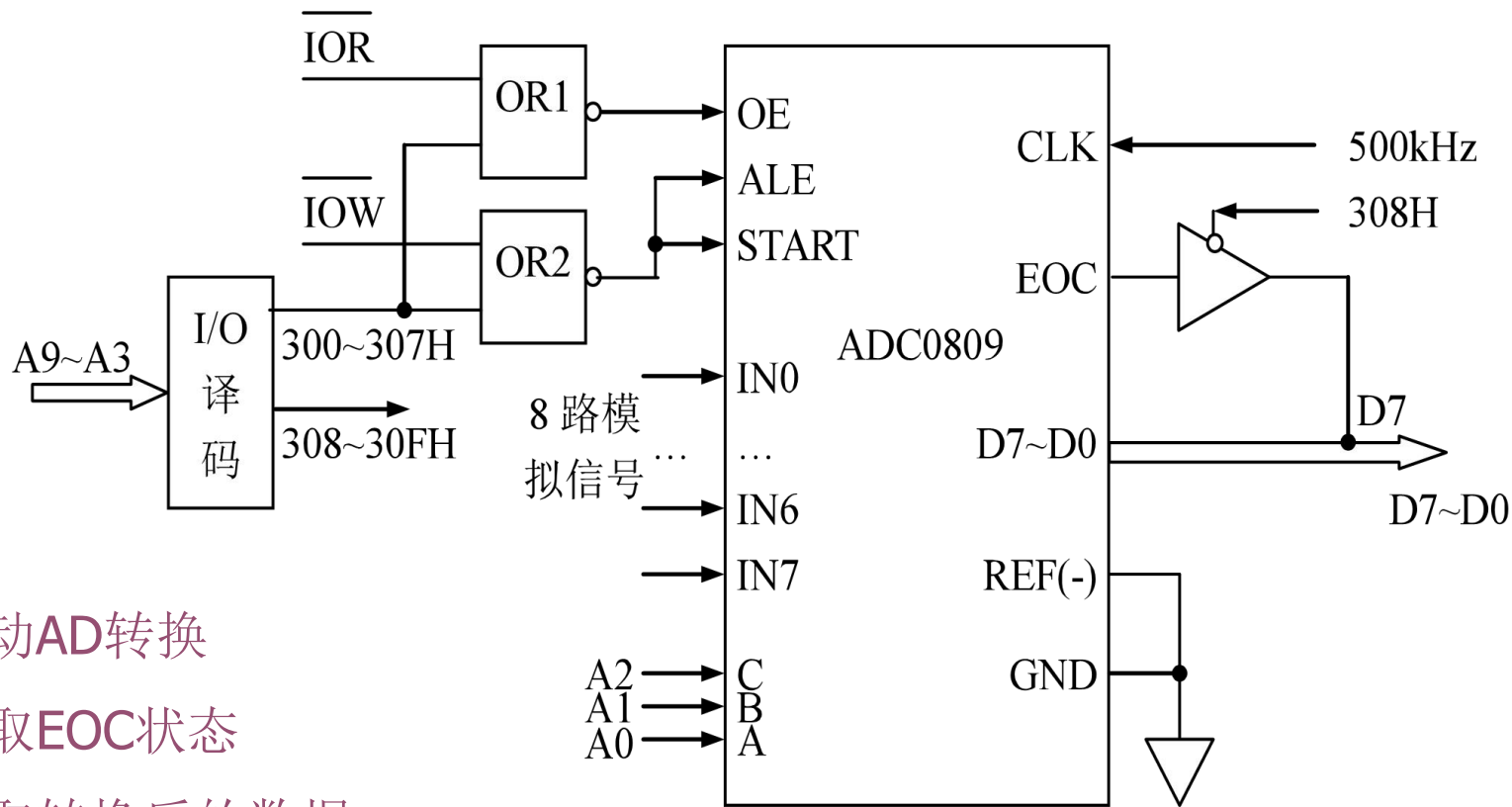
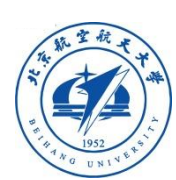
- 以200Hz的频率对每个通道均采集1024个数据（即每隔5ms对各通道轮流采集一个数据），存放到数据段中以DBUF为起始地址的数据缓冲区中。
- 数据存放次序与通道号一致，即从通道0开始，先依次存入每个通道的第一个数据，再存入各通道的第二个数据，直到各通道都存满1024个数据为止。

方案：选用8253芯片来产生定时脉冲，控制采样率

- 设8253通道0工作在方式2，CLK0时钟频率1MHz，端口基地址318H。
- 由于采样率为200Hz，所以时间常数为 $1\text{MHz}/200\text{Hz}=5000$ 。这样每隔5ms就会在8253的OUT0端输出一个脉冲

- 该脉冲加到PC机8259A的IR2引脚上，在8259A控制下定时向CPU发出中断请求。
- 中断服务程序中对8个引脚依次采集数据。中断服务程序执行1024次后，程序结束。
 - 用OUT指令启动转换
 - 然后查询EOC引脚的状态。当EOC状态高->低->高时，表示转换结束
 - 用IN指令读入结果。
 - 轮流启动各通道的转换并读取数据，存入数据缓冲区，就完成一次扫描。





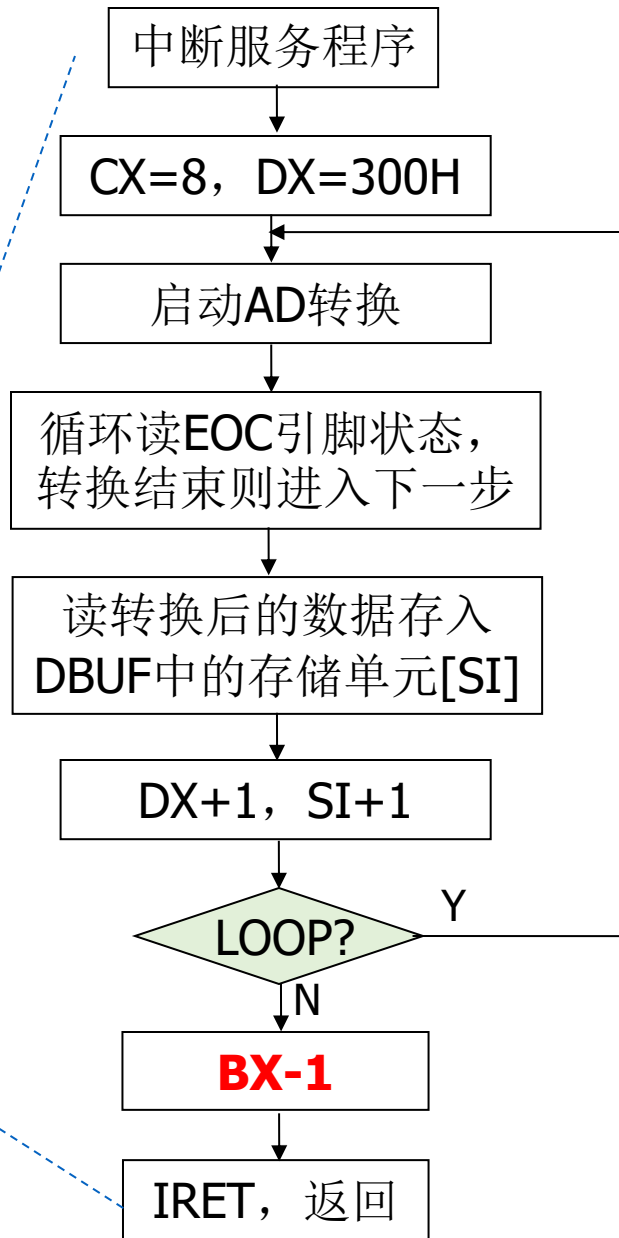
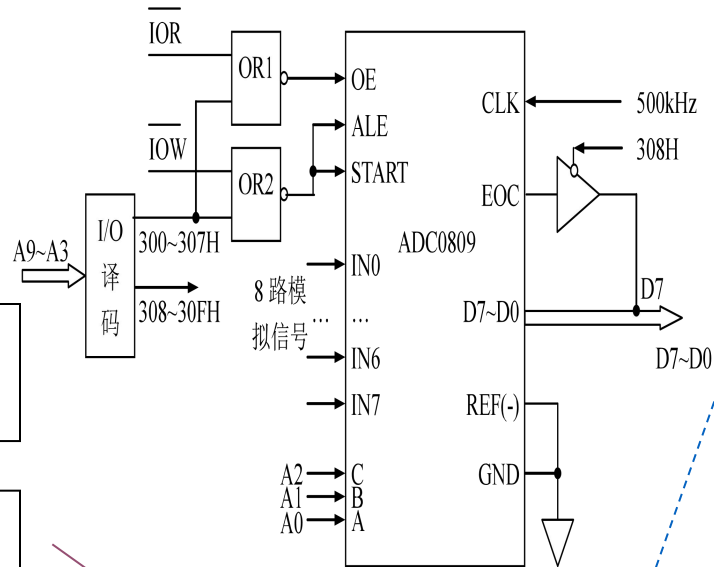
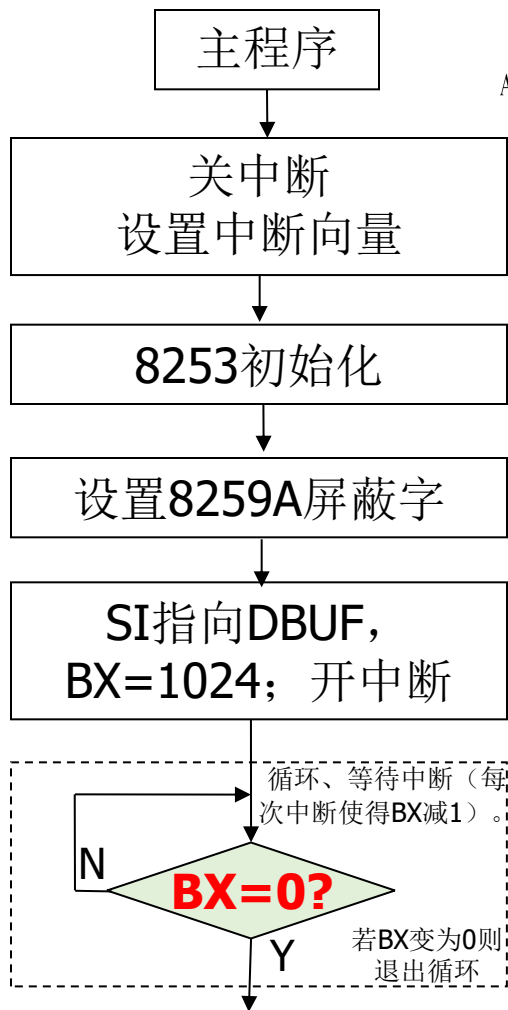
1.启动AD转换

2.读取EOC状态

3.读取转换后的数据

图 ADC0809多路数据采集电路

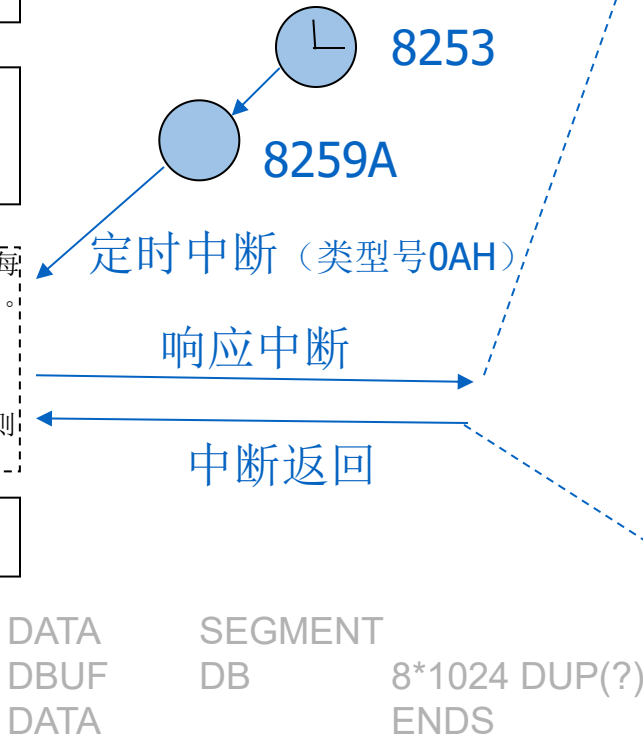
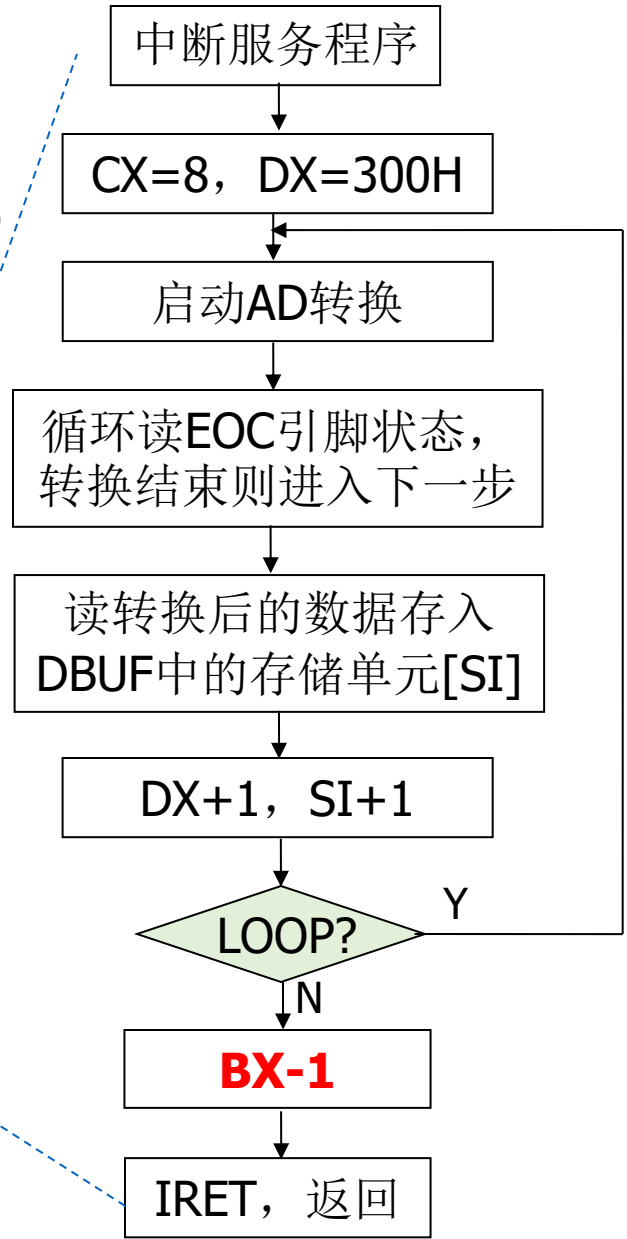
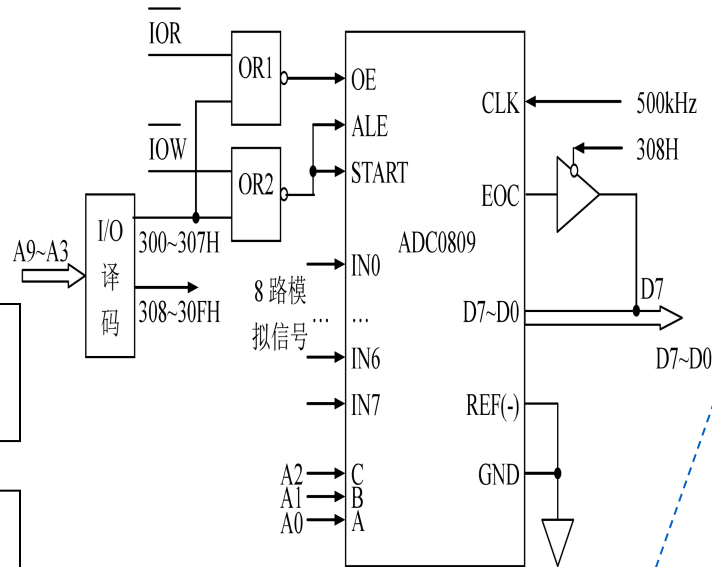
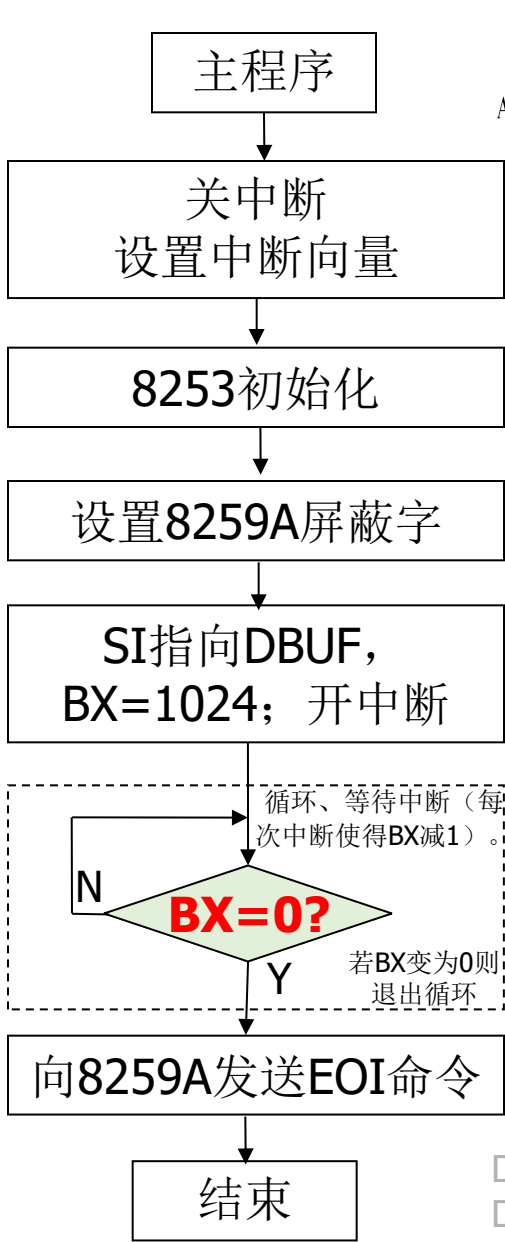
■ 流程图



DATA
DBUF
DATA

SEGMENT
DB
8*1024 DUP(?)
ENDS

流程图



DATA
DBUF
DATA

SEGMENT
DB
8*1024 DUP(?)
ENDS

程序

DATA	SEGMENT	;数据段
DBUF	DB 8*1024 DUP(?)	;数据区 (8×1024字节)
DATA	ENDS	
CODE	SEGMENT	;代码段
	ASSUME CS:CODE, DS:DATA	
ADMN	PROC FAR	;主程序
	MOV AX, DATA	
	MOV DS, AX	;DS指向数据段基址
	;设置0AH号中断向量	
	CLI	
	MOV AX, 0	
	MOV ES, AX	;
	MOV DI, 4*0AH	;使ES:DI=0000:(4*0AH)
	MOV AX, OFFSET ADINT	;AX=中断服务子程序偏移地址
	MOV ES:[DI], AX	;放入中断矢量表
	MOV AX, SEG ADINT	;取中断矢量段基址
	MOV ES:[DI+2], AX	;放入中断矢量表

;对8253进行初始化，使通道0的控制字为：方式2，只写高字节

;BCD计数，定时时间常数为5000

MOV	DX, 31EH	;DX指向8253控制寄存器
MOV	AL, 00100101B	;通道0控制字
OUT	DX, AL	;输出控制字

MOV	DX, 318H	;DX指向8253通道0
MOV	AL, 50H	;时间常数
OUT	DX, AL	;只写高8位，低8位自动清零

;对8259A设置屏蔽字，仅允许8259A的IR2和键盘中断，其余禁止

MOV	AL, 1111 1001B	;屏蔽字
OUT	21H, AL	;向屏蔽寄存器输出屏蔽字

;设置数据缓冲区起始地址到SI，计数初值到BX中，等待中断

;每通道采集完1024个数后结束中断

MOV SI, OFFSET DBUF

;SI指向数据缓冲区起始地址

MOV BX, 1024

;BX中存数据计数器初值

STI

;开中断，等待中断

AGAIN: CMP BX, 0

;每中断一次BX-1

JNZ AGAIN

;BX≠0，未采完，循环等待中断

MOV AL, 20H

;采完，送中断结束命令EOI→AL

OUT 20H, AL

;输出中断结束命令到8259A

MOV AH, 4CH

;退出

INT 21H

RET

;从子程序返回

ADMP ENDP

;ADMP过程结束

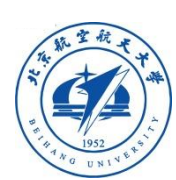
;

;中断服务程序，对每个通道均采集一个数据，存进DBUF

ADINT	PROC	NEAR	
	MOV	CX, 8	;设置通道计数器初值
	MOV	DI, 300H	;ADC通道0端口地址
NEXT:	MOV	DX, DI	;
	OUT	DX, AL	;启动一次转换
	MOV	DX, 308H	;DX指向状态口308H
POLL:	IN	AL, DX	;读入EOC状态
	TEST	AL, 80H	;EOC(D7)=0?即开始转换了?
	JNZ	POLL	;非0, 循环等待
NOEND:	IN	AL, DX	;EOC=0, 已经开始转换
	TEST	AL, 80H	;再查EOC是否为1
	JZ	NOEND	;EOC=0, 等待转换结束
	MOV	DX, 300H	;
	IN	AL, DX	;读取结果
	MOV	[SI], AL	;存储到缓冲区
	INC	DI	;DI指向下一个通道
	INC	SI	;地址指针指向下一个存储单元
	LOOP	NEXT	;循环8次, 采集8个通道



DEC	BX	;本轮采集结束, 计数器BX减1
STI		;开放中断
IRET		;从中断返回
ADINT	ENDP	
CODE	ENDS	
END		



总结

- 中断的基本概念
 - 中断优先级及其实现
- 8086中断系统
- 中断服务程序的设计
- 8259A